

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:
Leopoldo Bolaños
Matias Medina
Martin Palacios

Fecha: 2026/06/19

REQ004 (Servicios TAG):

Módulo que registra de forma automatizada trámites de peajes seleccionados desde un menú desplegable, asignando de manera exacta su tarifa correspondiente según el catálogo establecido.

1. CÓDIGO FUENTE para Servicios TAG

- Controlador (Capturas)

```
1 package espe.edu.ec.programacaja.controller;
2
3 import espe.edu.ec.programacaja.model.CashFlowModel;
4 import espe.edu.ec.programacaja.view.TagServicePanel;
5
6 import javax.swing.*;
7 import java.util.LinkedHashMap;
8 import java.util.Map;
9
10 public class TagServiceController {
11
12     private TagServicePanel view;
13     private CashFlowModel cashFlowModel;
14
15     private Map<String, Double> tagCatalog;
16     private Map<String, Integer> temporalServices;
17
18     public TagServiceController(TagServicePanel view, CashFlowModel cashFlowModel) {
19         this.view = view;
20         this.cashFlowModel = cashFlowModel;
21
22         tagCatalog = new LinkedHashMap<>();
23         temporalServices = new LinkedHashMap<>();
24
25         initCatalog();
26         initView();
27         initEvents();
28     }
29
30     private void initCatalog() {
31         tagCatalog.put("TAG", 10.00);
32         tagCatalog.put("Panavial", 5.00);
33         tagCatalog.put("Guayasamín", 5.00);
34         tagCatalog.put("Activación", 3.00);
35         tagCatalog.put("Desactivación", 3.00);
36     }
37 }
```

```

37
38 private void initView() {
39     String[] servicios = tagCatalog.keySet().toArray(new String[0]);
40     view.cargarServicios(servicios);
41     actualizarPrecio();
42     actualizarResumenTemporal();
43 }
44
45 private void initEvents() {
46     view.getCbServicioTag().addActionListener(e -> actualizarPrecio());
47
48     view.getBtnAgregar().addActionListener(e -> agregarServicioTemporal());
49
50     view.getBtnGuardarServicios().addActionListener(e -> guardarServiciosEnCaja());
51 }
52
53 private void actualizarPrecio() {
54     String servicio = view.getServicioSeleccionado();
55
56     if (servicio != null && tagCatalog.containsKey(servicio)) {
57         double precio = tagCatalog.get(servicio);
58         view.setPrecioTexto(String.format("%.2f", precio));
59     } else {
60         view.setPrecioTexto("$0.00");
61     }
62 }
63
64 private void agregarServicioTemporal() {
65     String servicio = view.getServicioSeleccionado();
66     int cantidad = view.getCantidad();
67
68     if (servicio == null || servicio.isEmpty()) {
69         showError("Debe seleccionar un servicio TAG.");
70         return;
71     }
72
73     if (cantidad <= 0) {
74         showError("La cantidad debe ser mayor a cero.");
75         return;
76     }
77
78     int cantidadActual = temporalServices.getOrDefault(servicio, 0);
79     temporalServices.put(servicio, cantidadActual + cantidad);
80
81     view.reiniciarCantidad();
82     actualizarResumenTemporal();
83 }
84
85 private void guardarServiciosEnCaja() {
86     if (temporalServices.isEmpty()) {
87         showError("No hay servicios TAG para guardar.");
88         return;
89     }
90
91     try {
92         for (Map.Entry<String, Integer> entry : temporalServices.entrySet()) {
93             String servicio = entry.getKey();
94             int cantidad = entry.getValue();
95             double precio = tagCatalog.get(servicio);
96
97             cashFlowModel.addTagServiceSale(servicio, cantidad, precio);
98         }
99
100         temporalServices.clear();
101         actualizarResumenTemporal();
102
103         JOptionPane.showMessageDialog(
104             view,
105             "Servicios TAG guardados correctamente en el cierre de caja.",
106             "Servicios guardados",
107             JOptionPane.INFORMATION_MESSAGE

```

```

110     } catch (IllegalArgumentException ex) {
111         showError(ex.getMessage());
112     }
113 }
114
115 private void actualizarResumenTemporal() {
116     if (temporalServices.isEmpty()) {
117         view.setResumenTemporal("No hay servicios TAG agregados.");
118         return;
119     }
120
121     StringBuilder sb = new StringBuilder();
122     double subtotal = 0.0;
123
124     sb.append("SERVICIOS TAG TEMPORALES\n\n");
125
126     for (Map.Entry<String, Integer> entry : temporalServices.entrySet()) {
127         String servicio = entry.getKey();
128         int cantidad = entry.getValue();
129         double precio = tagCatalog.get(servicio);
130         double total = precio * cantidad;
131
132         subtotal += total;
133
134         sb.append(String.format(
135             "%s $%.2f x%d = $%.2f\n",
136             servicio,
137             precio,
138             cantidad,
139             total
140         ));
141     }
142
143     sb.append(String.format("\nSubtotal temporal TAG: $%.2f", subtotal));
144
145     view.setResumenTemporal(sb.toString());

```

```

141     }
142
143     sb.append(String.format("\nSubtotal temporal TAG: $%.2f", subtotal));
144
145     view.setResumenTemporal(sb.toString());
146 }
147
148 private void showError(String message) {
149     JOptionPane.showMessageDialog(
150         view,
151         message,
152         "Error",
153         JOptionPane.ERROR_MESSAGE
154     );
155 }
156 }

```

Controlador (Código)

```
package espe.edu.ec.programacaja.controller;

import espe.edu.ec.programacaja.model.CashFlowModel;
import espe.edu.ec.programacaja.view.TagServicePanel;

import javax.swing.*.*;
import java.util.LinkedHashMap;
import java.util.Map;

public class TagServiceController {

    private TagServicePanel view;
    private CashFlowModel cashFlowModel;

    private Map<String, Double> tagCatalog;
    private Map<String, Integer> temporalServices;

    public TagServiceController(TagServicePanel view, CashFlowModel cashFlowModel) {
        this.view = view;
        this.cashFlowModel = cashFlowModel;

        tagCatalog = new LinkedHashMap<>();
        temporalServices = new LinkedHashMap<>();

        initCatalog();
        initView();
        initEvents();
    }

    private void initCatalog() {
        tagCatalog.put("TAG", 10.00);
        tagCatalog.put("Panavial", 5.00);
        tagCatalog.put("Guayasamín", 5.00);
        tagCatalog.put("Activación", 3.00);
        tagCatalog.put("Desactivación", 3.00);
    }

    private void initView() {
        String[] servicios = tagCatalog.keySet().toArray(new String[0]);
        view.cargarServicios(servicios);
        actualizarPrecio();
        actualizarResumenTemporal();
    }

    private void initEvents() {
        view.getCbServicioTag().addActionListener(e -> actualizarPrecio());

        view.getBtnAgregar().addActionListener(e -> agregarServicioTemporal());

        view.getBtnGuardarServicios().addActionListener(e -> guardarServiciosEnCaja());
    }

    private void actualizarPrecio() {
        String servicio = view.getServicioSeleccionado();
```

```

        if (servicio != null && tagCatalog.containsKey(servicio)) {
            double precio = tagCatalog.get(servicio);
            view.setPrecioTexto(String.format("%.2f", precio));
        } else {
            view.setPrecioTexto("$0.00");
        }
    }

private void agregarServicioTemporal() {
    String servicio = view.getServicioSeleccionado();
    int cantidad = view.getCantidad();

    if (servicio == null || servicio.isEmpty()) {
        showError("Debe seleccionar un servicio TAG.");
        return;
    }

    if (cantidad <= 0) {
        showError("La cantidad debe ser mayor a cero.");
        return;
    }

    int cantidadActual = temporalServices.getOrDefault(servicio, 0);
    temporalServices.put(servicio, cantidadActual + cantidad);

    view.reiniciarCantidad();
    actualizarResumenTemporal();
}

private void guardarServiciosEnCaja() {
    if (temporalServices.isEmpty()) {
        showError("No hay servicios TAG para guardar.");
        return;
    }

    try {
        for (Map.Entry<String, Integer> entry : temporalServices.entrySet()) {
            String servicio = entry.getKey();
            int cantidad = entry.getValue();
            double precio = tagCatalog.get(servicio);

            cashFlowModel.addTagServiceSale(servicio, cantidad, precio);
        }

        temporalServices.clear();
        actualizarResumenTemporal();

        JOptionPane.showMessageDialog(
            view,
            "Servicios TAG guardados correctamente en el cierre de caja.",
            "Servicios guardados",
            JOptionPane.INFORMATION_MESSAGE
        );
    } catch (IllegalArgumentException ex) {
        showError(ex.getMessage());
    }
}

```

```

    }
}

private void actualizarResumenTemporal() {
    if (temporalServices.isEmpty()) {
        view.setResumenTemporal("No hay servicios TAG agregados.");
        return;
    }

    StringBuilder sb = new StringBuilder();
    double subtotal = 0.0;

    sb.append("SERVICIOS TAG TEMPORALES\n\n");

    for (Map.Entry<String, Integer> entry : temporalServices.entrySet()) {
        String servicio = entry.getKey();
        int cantidad = entry.getValue();
        double precio = tagCatalog.get(servicio);
        double total = precio * cantidad;

        subtotal += total;

        sb.append(String.format(
            "%s $%.2f x%d = $%.2f\n",
            servicio,
            precio,
            cantidad,
            total
        ));
    }

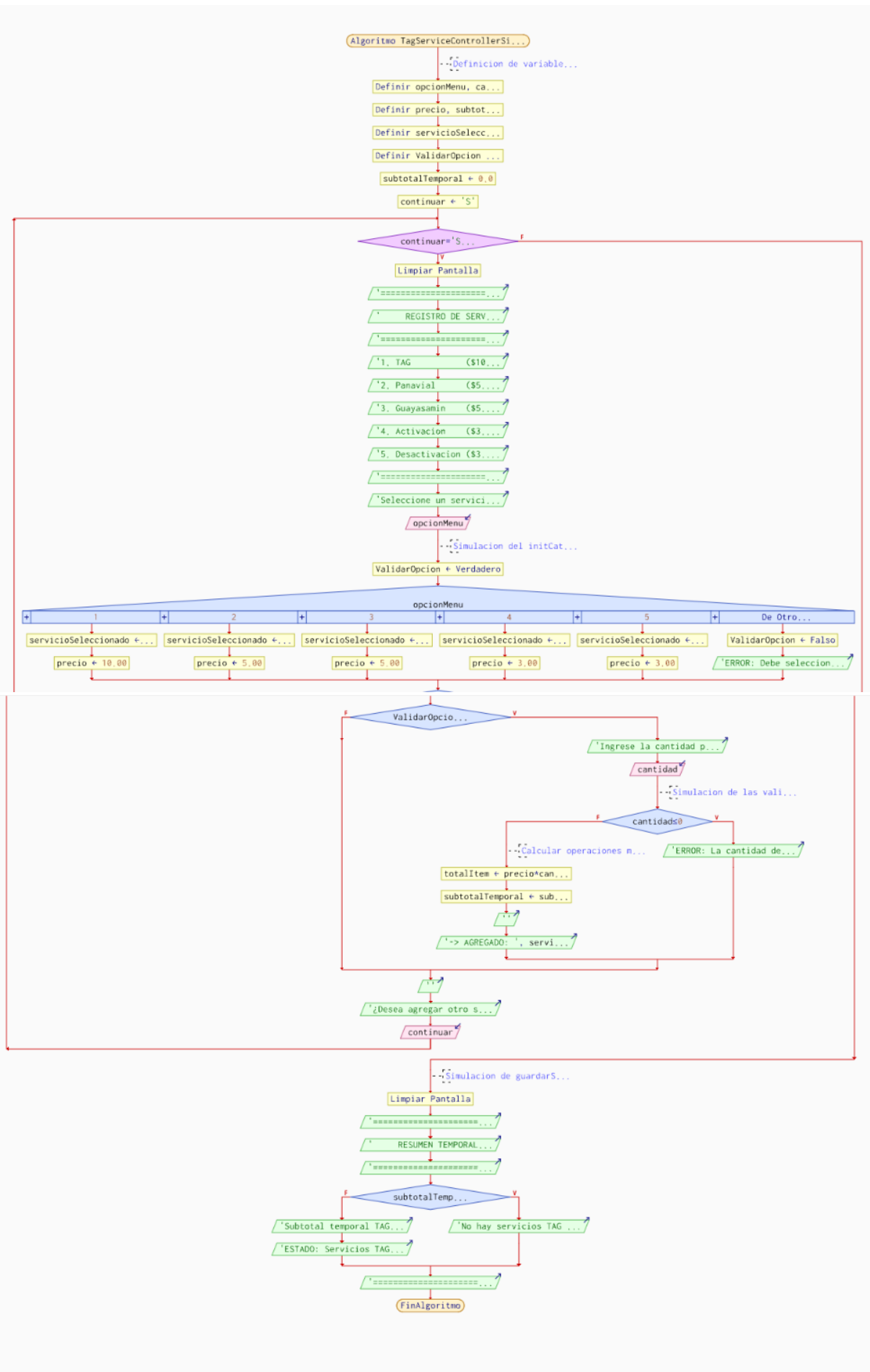
    sb.append(String.format("\nSubtotal temporal TAG: $%.2f", subtotal));

    view.setResumenTemporal(sb.toString());
}

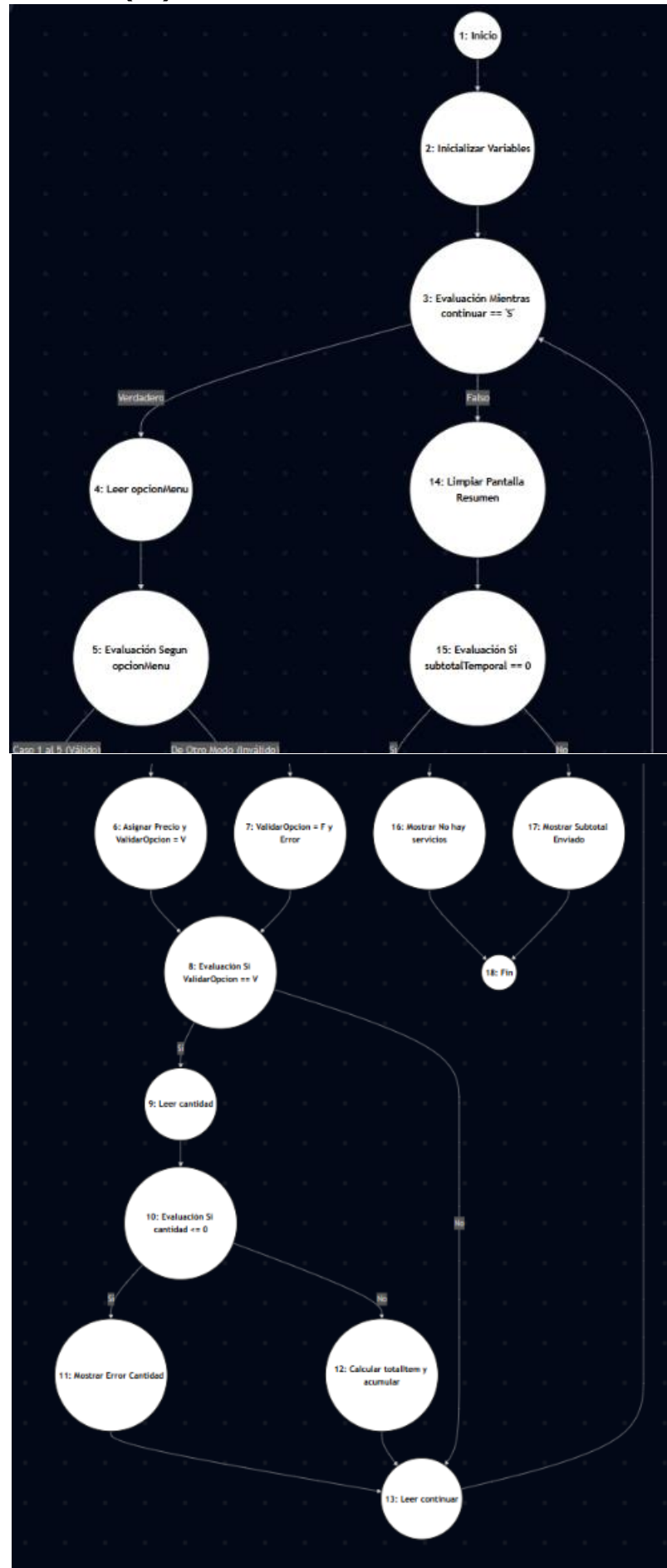
private void showError(String message) {
    JOptionPane.showMessageDialog(
        view,
        message,
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
}
}

```

2. DIAGRAMA DE FLUJO (DF) Servicios TAG



3. GRAFO DE FLUJO (GF) Servicios TAG



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Servicios TAG

Determinar en base al GF del numeral 4

RUTAS

R1(Camino de Salida Directa / Carrito Vacío): 1 - 2 - 3 [Falso] - 14 - 15 [Sí] - 16 - 18

R2 (Catálogo Inválido / Error de Selección): 1 - 2 - 3 [Verdadero] - 4 - 5 [De Otro Modo] - 7 - 8 [No] - 13 - 3 [Falso] - 14 - 15 [Sí] - 16 - 18

R3 (Cantidad Inválida / Error Menor o Igual a Cero): 1 - 2 - 3 [Verdadero] - 4 - 5 [Caso 1-5] - 6 - 8 [Sí] - 9 - 10 [Sí] - 11 - 13 - 3 [Falso] - 14 - 15 [Sí] - 16 - 18

R4 (Flujo Exitoso / Registro y Cierre Correcto): 1 - 2 - 3 [Verdadero] - 4 - 5 [Caso 1-5] - 6 - 8 [Sí] - 9 - 10 [No] - 12 - 13 - 3 [Falso] - 14 - 15 [No] - 17 - 18

5. COMPLEJIDAD CICLOMÁTICA Servicios TAG

Se puede calcular de las siguientes formas:

V/G) Nodos predicados (IF-THEN-ELSE) +1

$V(G) = 4 + 1$

$V(G) = 5$

$V(G) = A - N + 2$

$V(G) = 21 - 18 + 2$

$V(G) = 5$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos